IA Aplicada com LLMs

Aula 13 — CI e CD

Objetivos

Ao final deste deck, espera-se que o aluno seja capaz de:

- **explicar** a integração contínua como prática de equipe e **estimar** por que o custo da integração cresce mais que linearmente com o tempo de divergência;
- **distinguir** entrega de implantação contínua pelo ponto da decisão humana, e implantar de liberar;
- **descrever** a anatomia de um *pipeline* e **ordenar** os portões pelo custo e pela probabilidade de rejeição;
- **adaptar** o *pipeline* a um agente com LLM — testes determinísticos, *eval* com taxa mínima, chave como *secret* — e **quantificar** o poder de um *eval* pequeno;
- **ler** o workflow de `05-agent-ci/` e **situar** Jenkins, GitHub Actions, GitLab CI/CD, CircleCI e Argo CD.

Parte 1

Integração contínua

A prática antes da ferramenta

Integração contínua (*continuous integration*, CI): integrar as mudanças ao ramo principal com frequência, verificando cada integração com *build* e testes automáticos, de modo que o ramo principal permaneça implantável (FOWLER, 2006). O servidor de CI é só o executor de hábitos da equipe (DUVALL; MATYAS; GLOVER, 2007):

Hábito	O que exige
repositório único e *build* automatizado	tudo versionado; um único comando, a partir de um *checkout* limpo, produz o artefato
build que se testa	o mesmo comando executa a suíte; falha de teste falha o *build*
integrar todo dia, verificar cada integração	cada *commit* no ramo principal dispara o *build* numa máquina neutra
consertar primeiro	*build* quebrado no ramo principal é a prioridade: corrige-se em minutos ou reverte-se
build rápido, resultado visível	poucos minutos de verificação; o estado do ramo principal é público

Desenvolvimento baseado no tronco (*trunk-based development*): ramos de horas ou de poucos dias, integrados por mudanças pequenas. Ramos de semanas, cada um com seu servidor de CI, têm a ferramenta e não a prática: a integração continua rara e em lote.

O tronco vermelho para a linha, e a integração tardia custa ao quadrado

A regra do *build* quebrado — a segunda maneira do DevOps aplicada ao tronco: enquanto ele está vermelho, toda integração seguinte parte de uma base defeituosa e uma segunda falha se mistura à primeira. A correção do tronco precede trabalho novo, e reverter o *commit* responsável é correção legítima.

O custo da integração tardia. Dois ramos divergem por um intervalo T, cada um acumulando mudanças à taxa λ. Cada mudança de um lado pode conflitar — textual ou semanticamente — com cada mudança do outro:

$$C(T) = (\lambda T)^2$$

- estimativa de ordem de grandeza: a fração de pares que conflita depende do acoplamento, mas a forma não depende da constante;
- dobrar o intervalo entre integrações **quadruplica** os pares a examinar — o "inferno da integração" (*integration hell*);
- integrar várias vezes ao dia mantém $\lambda T \approx 1$: o conflito envolve uma única mudança recente.

Limite. A CI garante o que a suíte verifica, e nada além. Tronco verde com suíte fraca é implantável só no sentido formal.

Parte 2

Entrega e implantação contínuas

Onde fica a decisão humana; implantar não é liberar

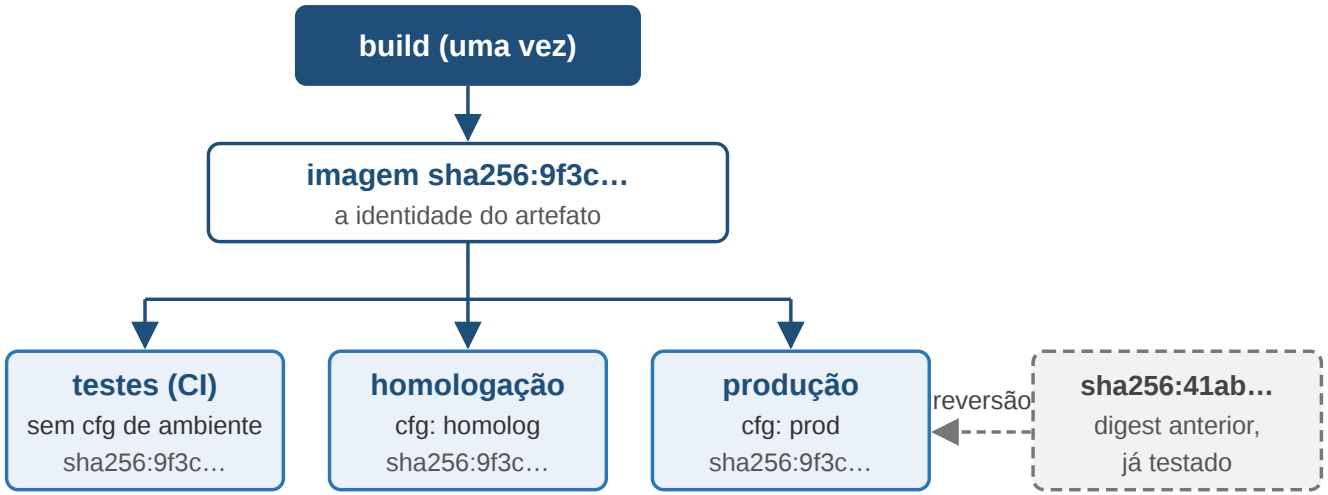
	Entrega contínua (*continuous delivery*)	Implantação contínua (*continuous deployment*)
depois do último portão	o artefato **pode** ir para produção	o artefato **vai** para produção
decisão humana	**quando** implantar — negócio, calendário, risco	nenhuma: os portões automáticos são a única barreira
o que elimina	o trabalho manual e o risco de executar a implantação	também a decisão

(HUMBLE; FARLEY, 2010)

Operação	Definição
implantar (*deploy*)	colocar uma versão do artefato em execução num ambiente
liberar (*release*)	expor um comportamento novo a usuários

As duas se separam por uma **chave de funcionalidade** (*feature flag*): o código é implantado desligado e liberado depois, a uma fração dos usuários ou a todos. *Canary*, *shadow* e teste A/B, da nota sobre MLOps, são formas de liberar gradualmente o que já está implantado. Resultado: implantação contínua com liberação controlada.

O artefato é construído uma vez e promovido



o mesmo digest em todos os ambientes; muda só a configuração

- Os estágios seguintes ao *build* **promovem** o mesmo artefato, identificado pelo mesmo *digest*; entre ambientes muda só a configuração injetada na execução (HUMBLE; FARLEY, 2010).
- Reconstruir em cada ambiente reabre a diferença: dependência resolvida de outro modo, imagem base atualizada — o testado deixa de ser o implantado. A **reversão** é promover um *digest* anterior, já testado.
- **Limite**: um *pipeline* que termina num artefato que ainda exige passo manual — migração à mão, configuração ajustada no servidor — automatiza a parte fácil e deixa a arriscada de fora.

Parte 3

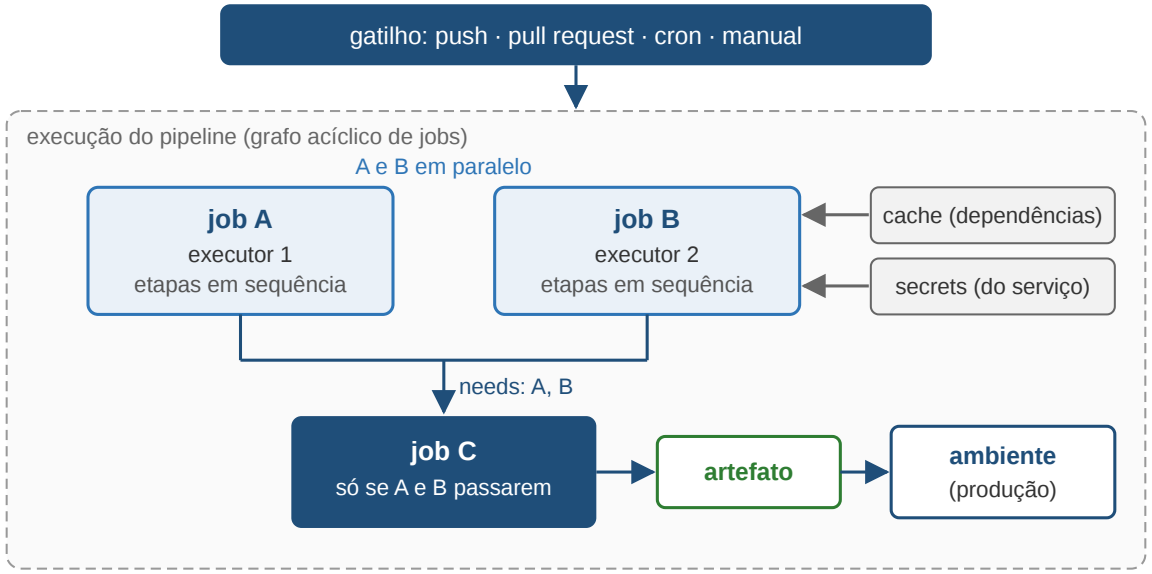
Anatomia de um *pipeline*

Os elementos de um *pipeline*

Um ***pipeline*** de CI/CD é um grafo acíclico dirigido de unidades de trabalho, declarado num arquivo versionado com o projeto e executado por um serviço a cada evento que o dispara.

Elemento	Definição
gatilho (*trigger*)	*push*, *pull request*, agendamento (*cron*), disparo manual, fim de outro *pipeline*
job	conjunto de etapas num mesmo executor, com sistema de arquivos próprio
etapa (*step*)	comando ou ação reutilizável, em sequência; a falha de uma falha o *job*
executor (*runner*, *agent*)	máquina ou *container* que executa o *job*, hospedado pelo serviço ou pela organização
dependência	aresta do grafo: o *job* só começa quando os anteriores passam; sem aresta, executam em paralelo
artefato	arquivo produzido por um *job* e guardado para outro ou para *download*
cache	diretório preservado entre execuções, indexado em geral pelo *hash* do arquivo de dependências
secret	valor sensível guardado pelo serviço, entregue como variável de ambiente e mascarado nos *logs*
ambiente (*environment*)	destino de implantação nomeado, com *secrets* e regras de proteção próprios

Um *pipeline* em execução



- **Artefato × *cache***: o artefato **transporta** o resultado adiante e precisa ser exato; o *cache* **acelera** a próxima execução e pode ser descartado sem mudar o resultado. Transportar resultado pelo *cache* reintroduz a dependência de estado que o *pipeline* elimina.
- **O executor é ponto de confiança**: recebe o código e os *secrets* e executa o que o arquivo manda. Num executor da organização compartilhado entre repositórios, um *job* malicioso pode deixar algo para o seguinte; os hospedados são, em geral, descartados ao fim de cada *job*.

Parte 4

Os portões e a ordem

A ordem dos portões minimiza o custo esperado

Portão (*gate*): *job* que pode reprovar a mudança. Ordem usual:

1. **análise estática e de segurança** — *lint* do código e do `Dockerfile`, varredura de configuração e dependências: segundos;
2. **testes unitários** — sem rede nem serviço externo: segundos;
3. **testes de integração** — banco, fila ou API reais ou em *container*: minutos;
4. **aceitação e *eval*** — ponta a ponta contra critérios de negócio: minutos a horas e, num agente, *tokens*;
5. **publicação ou implantação**.

Com n portões independentes, de custo c_i e probabilidade p_i de rejeitar uma mudança defeituosa, com parada no primeiro que rejeita:

$$E[C] = \sum_{i=1}^{n} c_i \prod_{j<i} (1 - p_j)$$

Trocar os adjacentes i e $i+1$ reduz o custo se e somente se $c_{i+1}\,p_i < c_i\,p_{i+1}$; a ordem ótima é a **crescente de** c_i/p_i. Um *lint* de 2 s que pega um quinto dos defeitos ($c/p = 10$ s) vem antes de um *eval* de 5 min que pega metade ($c/p = 600$ s).

Paralelismo e *shift left*

- **Tempo de retorno**: com c_i tomado como a duração do portão, a mesma ordem entrega a reprovação no menor tempo, ainda com o contexto da mudança.
- **Paralelismo**: portões sem dependência entre si executam juntos, e o retorno passa a ser o do mais lento do grupo, não a soma. Análise estática e testes unitários formam em geral o primeiro grupo; a publicação depende de todos.
- ***Shift left***: deslocar uma verificação para mais cedo no ciclo — da homologação para o *pipeline*, do *pipeline* para o *commit*, do *commit* para o editor. O custo de corrigir um defeito cresce com a distância entre sua introdução e sua detecção. A análise de segurança do `Dockerfile` no primeiro portão é *shift left* de uma verificação que, sem o *pipeline*, só ocorreria numa auditoria da imagem em produção.
- **Hipótese e limite**: a independência entre portões é simplificação — eles detectam defeitos sobrepostos —, mas produz a regra que a literatura recomenda: o rápido e barato primeiro (HUMBLE; FARLEY, 2010). A ordem otimiza o custo de rejeitar; não substitui portão ausente. Um defeito que nenhum portão detecta chega à produção pelo *pipeline* mais barato possível.

Parte 5

CI/CD para um agente com LLM

Duas classes de portão: testes determinísticos e *eval*

	Testes determinísticos	***Eval***
o que verificam	o código em volta do modelo: ferramentas, extração da resposta, validação da entrada, API	o comportamento do agente com o modelo real, contra um gabarito
o modelo	não é chamado; no fluxo que passa por ele, um **dublê** (*test double*) devolve resposta fixa	é chamado
custo e tempo	gratuitos, segundos, repetíveis	lento, consome *tokens*, não determinístico
critério	igualdade: o valor esperado ou falha	**taxa mínima de acerto** sobre o conjunto, com limiar declarado antes da execução
quando rodam	em todo *push*	quando há chave e o gatilho justifica o custo

O *eval* do *pipeline* é a execução automática da suíte de regressão construída na aula de *evals*, e herda o poder estatístico que ela tiver: cobertura, tamanho do conjunto, repetição de cada caso e limiar de promoção são decisões da suíte.

O poder de um *eval* pequeno é binomial

n casos, acerto q por caso (igual e independente entre casos), mínimo de k acertos:

$$P(\text{passa}) = \sum_{i=k}^{n} \binom{n}{i} q^i (1-q)^{n-i}$$

Com $n = 4$ e limiar de 75%, ou seja $k = 3$:

q **(acerto real por caso)**	$P(\text{passa})$	**Leitura**
0,9	$0{,}9^4 + 4 \cdot 0{,}9^3 \cdot 0{,}1 \approx 0{,}95$	reprovado em cerca de 1 execução a cada 20, sem mudança nenhuma
0,7	$0{,}7^4 + 4 \cdot 0{,}7^3 \cdot 0{,}3 \approx 0{,}65$	aprovado em cerca de 2 a cada 3

Com quatro casos, o portão detecta **regressões grosseiras** — o agente que deixou de chamar a ferramenta, o prompt que quebrou a extração do veredito — e **não distingue versões próximas**.

A chave do modelo é um *secret*

A chave do provedor é um *secret* do serviço de CI, entregue ao *job* do *eval* como variável de ambiente — nunca um arquivo do repositório. Consequências:

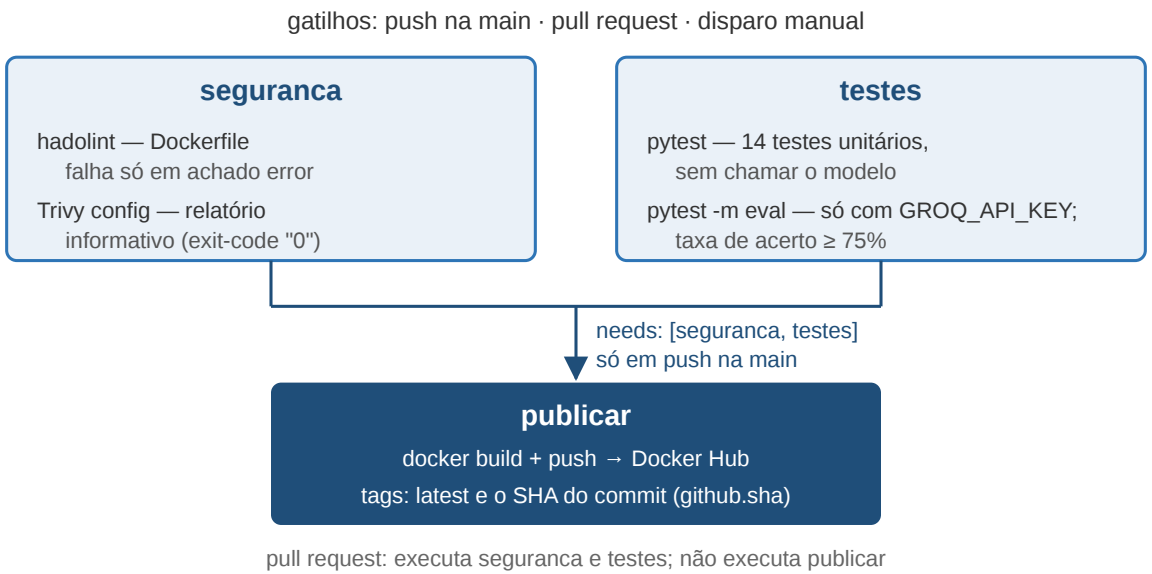
- **o *eval* é opcional quando a chave não existe**: um *fork*, ou uma execução sem o *secret*, roda os testes determinísticos e **pula** o *eval*, em vez de falhar por falta de credencial. No GitHub Actions, *secrets* não são entregues a workflows disparados por *pull request* vindo de *fork* (GITHUB, 2026); o mesmo *pipeline* funciona nos dois casos;
- **o custo é controlado pelo gatilho**: *eval* só no *push* para o tronco, só em mudanças no prompt ou na configuração do modelo, ou por agendamento — uma decisão de custo a declarar, não um detalhe do arquivo.

Limite. O *eval* mede o modelo no momento da execução. Um provedor que atualiza o modelo por trás de um nome sem versão muda o comportamento sem *commit*, e nenhum gatilho de *push* dispara: o **eval agendado** é a forma de detectar esse *drift* entre mudanças de código.

Parte 6

O caso: `05-agent-ci`

Três *jobs*: verificar, testar, publicar



`codigo/05-agent-ci/`: o agente como API da nota de *script* a imagem, com testes e o workflow `.github/workflows/ci.yml` do GitHub Actions.

O que cada *job* verifica, e quando falha

Job	O que verifica ou faz	Critério de falha
`seguranca`	**hadolint** lê o `Dockerfile` contra regras de boas práticas; **Trivy**, em modo de configuração, procura má configuração de segurança	hadolint só em achado de severidade *error*; Trivy informativo (`exit-code: "0"`)
`testes`	instala `requirements-dev.txt` com *cache* do pip; testes unitários de ferramentas, veredito e API; *eval* se o *secret* existir	qualquer teste unitário falho; acerto do *eval* abaixo de 75%
`publicar`	constrói a imagem e a envia ao Docker Hub com duas *tags*	executa só em *push* para a `main`, depois que os dois anteriores passam

- **Unitários** (`tests/test_ferramentas.py`, `tests/test_api.py`) não chamam o modelo: o `conftest.py` define uma chave falsa para que o cliente possa ser construído, e o teste da API troca `rodar` por um dublê via `monkeypatch`.
- **Eval** (`tests/test_eval.py`): quatro casos com veredito conhecido pela política, `TAXA_MINIMA = 0.75`. O marcador `eval` é excluído por padrão — `addopts = -m "not eval"` no `pytest.ini` —, e um `pytest` sem argumento não gasta *tokens*.
- `requirements-dev.txt` acrescenta pytest e httpx e **não entra na imagem**, que instala só o `requirements.txt`.

O workflow, em trechos

```yaml
on:
  push:
    branches: [main]
  pull_request:
  workflow_dispatch:

permissions:
  contents: read

jobs:
  seguranca: ...
  testes:
    runs-on: ubuntu-24.04
    env:
      GROQ_API_KEY: ${{ secrets.GROQ_API_KEY }}
    steps:
      ...
      - name: testes unitários
        run: pytest -v
      - name: eval do agente
        if: env.GROQ_API_KEY != ''
        env:
          LLM_PROVEDOR: groq
        run: pytest -v -s -m eval

  publicar:
    needs: [seguranca, testes]
    if: github.event_name == 'push' && github.ref == 'refs/heads/main'
    ...
          tags: |
            ${{ secrets.DOCKERHUB_USERNAME }}/agent-ci:latest
            ${{ secrets.DOCKERHUB_USERNAME }}/agent-ci:${{ github.sha }}
```

Resultados: comandos executados localmente, workflow não executado no GitHub

Os comandos equivalentes aos três *jobs*, em `comandos.md`, foram executados na máquina local; **o workflow não foi executado no GitHub**.

Verificação	Resultado
hadolint	passou, com um aviso: **DL3042** — `pip install` sem `--no-cache-dir`, o *cache* do pip fica na camada —, abaixo do limiar de falha
Trivy (configuração)	achado **AVD-DS-0002**, severidade alta: o `Dockerfile` não declara `USER`, e o processo roda como `root`
14 testes unitários	passaram, sem chamar o modelo
eval	passou, com o modelo real

O achado de `root` como portão opcional. O Trivy relata e não reprova, e o achado não impede a publicação. A escolha é reversível por uma linha: com `exit-code: "1"`, o mesmo achado barra `publicar`. Um portão que reprova sempre, por um achado conhecido e ainda não corrigido, acaba contornado ou desligado — e deixa de detectar o achado novo. A correção do `Dockerfile` e a leitura do arquivo como superfície de ataque estão na aula de segurança.

As decisões do arquivo, e o que ele não verifica

Decisão	Razão
tags `latest` e `github.sha`	`latest` é movida a cada publicação; a do SHA é escrita uma vez e liga a imagem ao *commit* exato — a linhagem da nota sobre MLOps, mecanizada. A implantação referencia a do SHA
`permissions: contents: read`	o *token* do GitHub em cada *job* só lê o repositório: um passo comprometido não escreve com ele
pull request sem `publicar`	a mudança é verificada antes de chegar ao tronco, e só o tronco produz imagem publicada
`DOCKERHUB_TOKEN`, não a senha	o token tem escopo restrito e é revogável sozinho

Lacunas em relação ao artefato promovido:

- **os testes não rodam na imagem publicada** — `testes` usa o Python do executor, e a imagem só é construída em `publicar`. Dependência de sistema ausente ou arquivo fora do `COPY` passam pelos testes e chegam ao registro. A forma estrita constrói a imagem primeiro, testa dentro dela e publica o mesmo *digest*;
- **não há implantação** — o *pipeline* termina no registro: integração contínua com entrega do artefato, não implantação contínua.

Parte 7

As ferramentas

Cinco ferramentas, uma anatomia

	Ferramenta	Modelo	*Pipeline* declarado em
	**Jenkins**	servidor de automação de código aberto, auto-hospedado; agentes ligados ao servidor; estende-se por *plugins*	`Jenkinsfile`, declarativo ou *script* Groovy (JENKINS, 2026)
	**GitHub Actions**	integrado ao GitHub; executores do GitHub ou da organização; etapas reutilizáveis como *actions*	YAML em `.github/workflows/` (GITHUB, 2026)
	**GitLab CI/CD**	integrado ao GitLab, hospedado ou auto-hospedado; GitLab Runner; *jobs* agrupados em estágios	`.gitlab-ci.yml` (GITLAB, 2026)
	**CircleCI**	hospedado, independente da plataforma do repositório; configuração reutilizável em *orbs*	`.circleci/config.yml` (CIRCLECI, 2026)
	**Argo CD**	implantação contínua para Kubernetes por GitOps: um agente no *cluster* reconcilia o estado declarado no git com o real	manifestos Kubernetes num repositório git (ARGO PROJECT, 2026)

Nas cinco, o *pipeline* é **código**: arquivo versionado, revisado com a mudança, com histórico e reversão — o controle de versão de tudo, da nota sobre DevOps, estendido ao processo de entrega. Migrar de uma para outra é tradução de sintaxe, não de conceito.

Push × *pull*, e a *action* fixada por SHA

	Push **— Jenkins, GitHub Actions, GitLab CI/CD, CircleCI**	***Pull*** **— Argo CD (GitOps)**
quem aplica	o *pipeline*, ao terminar, executa o comando que leva o artefato ao ambiente	o agente no *cluster* detecta a diferença entre o git e o real e a aplica
o que o *pipeline* faz	implanta	atualiza o repositório de estado — por exemplo, a *tag* da imagem num manifesto
credencial de escrita no ambiente	no serviço de CI	dentro do ambiente

Fixar uma *action*. O `05-agent-ci` referencia cada *action* por *tag* (`actions/checkout@v4`). A *tag* é um ponteiro que o dono do repositório da *action* pode mover; o SHA completo do *commit* é a única referência imutável, e é a forma recomendada pelo guia de segurança do GitHub Actions (GITHUB, 2026).

- **Precedente**: em março de 2025, as *tags* de `tj-actions/changed-files` foram movidas para um *commit* que imprimia os *secrets* nos *logs* (CVE-2025-30066); todo workflow que a referenciava por *tag* executou o código novo sem mudança no próprio repositório.
- O argumento é o da imagem base fixada até o *patch*: uma referência móvel transfere a quem a move a decisão sobre o que é executado. O custo é a atualização manual, que ferramentas de atualização de dependências abrem como *pull request*. A cadeia de suprimentos é desenvolvida na aula de segurança.